Mayalok: From a Smartphone Walkthrough to a Searchable Gaussian-Splat Digital Twin with Obstacle-Aware Browser Navigation

Ajay Yadav[1], Hitesh Bhatiwal[2], and Harsh Vardhan[3]
Team QWERTY (Team ID TM-E434D6B6C8)
Techfest 2026–27, Namma Space Challenge, IIT Bombay
[1]Team leader
Code and live demo: https://github.com/The-Harsh-Vardhan/mayalok, https://mayalok.vercel.app

***Abstract*—Digital twins of indoor venues are usually built with LiDAR scanners or CAD models, which most small venues cannot afford. This paper presents Mayalok (formerly Namma Space), an open pipeline and web viewer that turns ordinary smartphone video into a photorealistic 3D Gaussian Splat of an indoor space and serves it as a zero-install first-person walkthrough with place search and obstacle-aware walking directions. An offline, resumable Python pipeline extracts sharp keyframes, recovers camera poses with COLMAP-family structure-from-motion, fixes gravity, floor and metric scale (ArUco markers or a fixed phone-height rule), trains a 3D Gaussian Splatting model, removes floaters, and derives a 5 cm occupancy grid. The viewer renders the compressed `.spz` splat with Spark.js on Three.js, indexes places with Fuse.js, and plans routes with A* in a Web Worker over the same grid. We report two real runs on one incubation-centre capture. A single 56.8 s video produced 293 keyframes, of which 65 registered (6,660 sparse points, 0.52 px mean reprojection error); Brush trained 430,487 Gaussians, 191,378 of which survived cleaning, giving a 4.7 MB `scene.spz` and a 2.66 m^2 walkable area after agent-radius tuning. A second run over all 603 captured images registered 523 images in one GLOMAP model at 0.72 px, but the resulting splat showed cross-clip misalignment and was excluded. We present these results as a feasibility study, state what was not measured, and describe the limitations that follow.**

***Index Terms*—3D Gaussian Splatting, structure from motion, indoor digital twin, occupancy grid, A* pathfinding, web rendering, smartphone capture.**

I. Introduction

Indoor navigation aids for campuses, incubators, exhibitions, and small commercial venues depend on an accurate digital model of the space. Building such a model with terrestrial laser scanners or hand-authored CAD floor plans is slow and expensive, and the result is rarely photorealistic. At the same time, modern phones record sharp high-resolution video, and neural scene representations can turn such video into a navigable 3D scene. Neural Radiance Fields (NeRF) made photorealistic novel-view synthesis from posed images practical [1], [2], and 3D Gaussian Splatting (3DGS) added real-time rendering by representing a scene as millions of anisotropic Gaussians that are rasterized and alpha-blended [3]. The gap that remains is not reconstruction quality alone. A visitor needs a scene that loads in a browser, uses true metric units, can be searched for named places, and supports walking directions that respect walls and furniture.

Mayalok addresses that gap as an end-to-end system. It combines an offline reconstruction pipeline with a static web viewer, and it derives both collision and routing from a single 2D walkable grid computed from the splat itself. The central design decision is to treat that grid as the one spatial index shared by collision, pathfinding, and a future augmented-reality layer. A second decision is to keep every pipeline stage a plain-file transformation, so a stage can be re-run, tuned, or swapped without recomputing earlier ones.

This report documents the design and the evidence behind it. We deliberately limit the scope to what the repository implements and what was measured on one real capture: an incubation centre recorded on 24 Oct 2026 for the Techfest 2026–27 Namma Space challenge. We do not claim a benchmark against other capture systems, a user study, or sub-centimetre accuracy, and where the reference-style sections of a measurement paper would call for such results we instead state the method, the limitation, and the planned work.

A. Contributions

The main contributions are as follows:

- We present an open, resumable smartphone-to-splat pipeline with eight stages (frames, SfM, alignment, training, cleaning, navigation grid, export, viewer) and a metric, floor-at-zero, $+Y$-up world frame shared by every later stage.
- We describe a single-grid design in which a 5 cm occupancy grid derived from the cleaned splat drives collision, A* routing with wall-aware cost, path smoothing, and turn-by-turn instructions in a browser Web Worker.
- We report two real reconstruction runs of the same venue with exact counts, including a single-video run that produced a usable but partial scene and a multi-clip run that registered most images yet failed visually, and we analyze why low reprojection error was not sufficient evidence of a usable splat.

- We release the system as an MIT-licensed repository, a live demo, and a Python package (`pip install mayalok`, PyPI release 0.1.0) [4].

II. Related Work

A. Novel-View Synthesis and Gaussian Splatting

NeRF represents a scene as a neural field queried along camera rays and optimized from posed images [1]. Mip-NeRF 360 extended this to unbounded real scenes with anti-aliased sampling and established a widely used benchmark of indoor and outdoor captures [2]. 3DGS replaced the implicit field with an explicit set of Gaussians whose means, covariances, opacities, and view-dependent colors are optimized, and rendered them with a tile-based differentiable rasterizer at interactive rates [3]. Each Gaussian is

$$G(\mathbf{x}) = \exp\!\left(-\tfrac{1}{2}(\mathbf{x}-\boldsymbol{\mu})^{\top}\boldsymbol{\Sigma}^{-1}(\mathbf{x}-\boldsymbol{\mu})\right), \quad \boldsymbol{\Sigma} = \mathbf{R}\mathbf{S}\mathbf{S}^{\top}\mathbf{R}^{\top}, \tag{1}$$

and a pixel color is the front-to-back blend

$$C(\mathbf{p}) = \sum_{i\in\mathcal{N}} \mathbf{c}_i\,\alpha_i \prod_{j<i}(1-\alpha_j)\,. \tag{2}$$

Densification strategy and Gaussian count matter for deployment. A Markov-chain Monte Carlo formulation of densification allows a hard cap on the number of Gaussians, which bounds memory and download size [5]. The open-source library gsplat packages efficient 3DGS training with this strategy [6], and Brush provides a cross-platform trainer [7]. Mayalok uses both: gsplat is the default trainer and Brush is the fallback on machines where gsplat does not build.

B. Structure from Motion

3DGS needs camera poses, which are typically obtained with structure from motion. COLMAP is the standard incremental pipeline, built on SIFT features [8] and robust geometric verification [9]. Incremental mapping is accurate but slow and can fragment a capture into several sub-models. Global SfM, in particular GLOMAP, estimates all poses jointly and is much faster on large unordered sets [10]. Our pipeline uses the global mapper first and falls back to incremental mapping if too few frames register. The multi-view geometry underlying triangulation of markers and sparse points follows standard treatments [11].

C. Metric Scale, Fiducials, and Orientation

Monocular SfM recovers geometry only up to a similarity transform. A routing system needs true meters, so we recover scale from printed square fiducial markers detected with ArUco [12] or, when none are visible, from a fixed phone height. Gravity direction comes from camera orientations, the floor plane from a RANSAC fit [13], and in-plane rotation from a Manhattan-world assumption that walls align with two orthogonal axes [14].

D. Occupancy Grids and Pathfinding

Occupancy grids discretize free space for planning. A* finds minimum-cost paths on such a graph with a consistent heuristic [15]. We use an 8-connected grid with octile distance, add a clearance-dependent step cost so routes centre themselves in corridors, and smooth the cell path by string pulling. The grid is computed with standard array and morphology routines [16], [17].

E. Web Delivery of Splats

Browser delivery needs a compact format and a GPU renderer. The SPZ format compresses Gaussian splats for transmission [18], Spark.js renders splats inside Three.js scenes [19], [20], and splat-transform converts PLY files to SPZ [21]. For place lookup we use client-side fuzzy search [22]. Table I positions Mayalok against these building blocks; it is a qualitative comparison of roles, not a performance comparison.

III. Capture Data and Study Scope

A. Venue and Captures

The quantitative evidence comes from one venue, an incubation centre, captured on 24 Oct 2026 with a smartphone. The Technical Abstract records the session as 8 videos and about 40 photos. Two reconstruction runs are reported. The *single-video run* uses one 56.8 s portrait video. The *all-captures run* uses 603 images, which are keyframes from 7 videos together with 32 photos. The single-video run used a laptop with an RTX 4050 GPU (6 GB); the hardware of the multi-clip run is not recorded here. Table II records the scope. This boundary matters because the paper does not claim results across venues, devices, or lighting conditions.

B. Pipeline Stage Inventory

Table III lists the stages, the module that implements each, the method, and the durable output. Every stage reads and writes plain files, and the orchestrator skips a stage whose outputs already exist unless `--force` is given. This lets a failed or retuned stage resume without recomputing earlier ones.

IV. Methodology

A. System Overview

Mayalok has two halves. The offline half runs in Python and produces a self-contained scene folder (`scene.spz`, `scene.json`, `navgrid.json`, `minimap.png`, `pois.json`, `report.json`). The online half is a fully static site that loads that folder, renders the splat, and runs search and routing on the client. Figure 1 presents the conceptual architecture, Fig. 2 shows the system context, and Fig. 3 the stage outputs.

TABLE I
POSITIONING OF MAYALOK AGAINST THE COMPONENTS IT BUILDS ON (QUALITATIVE)

Work	Representation	Contribution	Role in Mayalok
NeRF [1], Mip-NeRF 360 [2]	Implicit radiance field	Photorealistic novel views; unbounded-scene benchmark	Motivation; not used, Mayalok adopts explicit splats instead
3DGS [3]	Explicit Gaussians	Real-time rasterized splat rendering	Core scene representation
3DGS-MCMC [5], gsplat [6]	Explicit Gaussians with capped count	Bounded Gaussian budget; training library	Default training path with 1.5M cap
COLMAP [9], GLOMAP [10]	Sparse points and poses	Incremental and global SfM	Pose recovery and fallback
ArUco [12]	Square fiducials	Robust marker detection	Optional metric-scale source and AR anchors
SPZ [18], Spark.js [19]	Compressed splat file, GPU renderer	Small downloads, in-browser rendering	Delivery and rendering layer
Mayalok	Splat plus 5 cm walkable grid and POI file	End-to-end capture, grid, search, and routing	Integrates the above into one resumable pipeline and static viewer

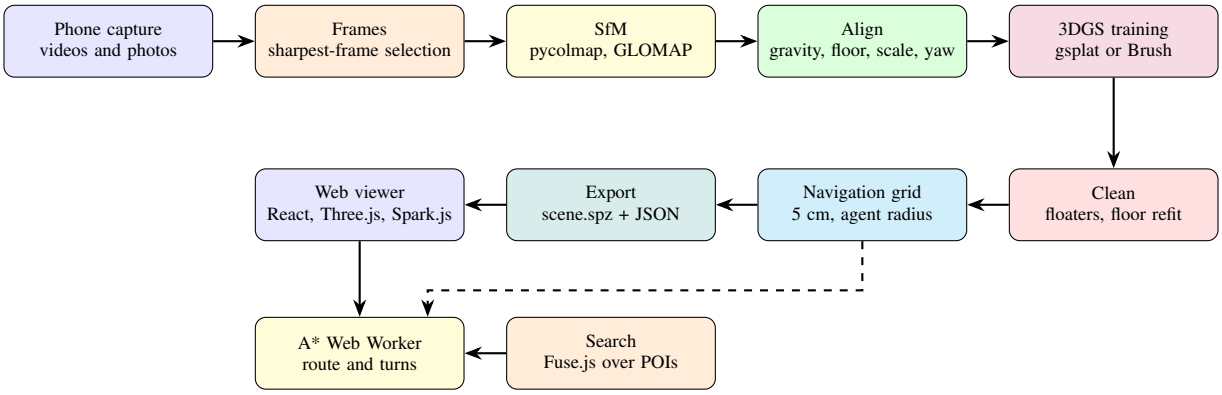


Fig. 1. Conceptual architecture of Mayalok. The offline pipeline (top row, then right to left on the second row) produces a scene folder. The viewer loads it, and the navigation grid (dashed) is posted to a Web Worker that plans routes for searched places.

TABLE II
DATA AND STUDY SCOPE

Item	Value
Venue	Incubation centre (one site)
Capture date	24 Oct 2026
Capture session	8 videos, about 40 photos
Single-video run input	1 portrait video, 56.8 s
All-captures run input	603 images (7 videos + 32 photos)
Compute (single-video run)	Laptop, RTX 4050 (6 GB)
Metric scale source	Phone-height rule (1.35 m); no ArUco markers in this capture
Challenge	Techfest 2026–27, Namma Space, IIT Bombay

B. Capture Protocol

The capture standard operating procedure (SOP) is part of the method, because reconstruction quality is bounded by capture quality. The operator turns all lights on, clears floor clutter, closes unwanted doors, and avoids mirrors and glass. Scale comes from 3–4 printed ArUco markers (`DICT_4X4_50`, 15 cm side) taped flat on the floor, or from a constant phone height of 1.35 m. The operator walks slowly (about one step per second), covers the perimeter and then an inner pass, photographs corners and ceiling edges, keeps 60–70% overlap, and walks the routes real visitors would use. One continuous clip suffices for a small space; a large space is split into one clip per zone with about 2 m of overlap. Capture takes about 3–5 minutes per room. The routes walked matter later, because the grid treats walked cells as free.

C. Keyframe Extraction

Videos are sampled at three times the target rate (default 2 keyframes per second after selection, long side capped at 1600 px). In each window only the sharpest frame is kept. Sharpness is the variance of the Laplacian of a downscaled grayscale copy,

$$S(I) = \mathrm{Var}\big(\nabla^2 I\big)\,, \tag{3}$$

and a window is discarded if its best score is below 35% of the median score over the clip. Photos are kept unless they are blurry. Each clip is written to its own folder so SfM can estimate separate intrinsics per clip. The single-video run used 5.5 keyframes per second at a 1024 px cap, which differs from the defaults and yielded 293 keyframes.

D. Structure from Motion

pycolmap extracts SIFT features (up to 8,192 per image) with a `SIMPLE_RADIAL` camera model and one set of intrinsics per clip. Each frame is matched against its next 15 neighbours, with quadratic long-range jumps, and with

TABLE III
PIPELINE STAGE INVENTORY

No.	Module	Method	Durable output
1	`namma.frames`	Oversampled extraction, sharpest frame per window (variance of Laplacian), blur floor at 35% of median	`images/<clip>/*.jpg`, `frames.json`
2	`namma.sfm`	SIFT features, sequential matching (15 neighbours), global mapper with incremental fallback	`sparse/`, undistorted dataset
3	`namma.align`, `namma.markers`	Gravity from cameras, RANSAC floor, scale from markers or phone height, Manhattan yaw	Metric `cameras.json`, `alignment.json`
4	`namma.train`	3DGS via gsplat (MCMC, 30k steps, 1.5M cap) or Brush fallback	Trained `point_cloud.ply`
5	`namma.clean`	Opacity, size, bounds and statistical-outlier filters; vertical floor refit	Cleaned splat
6	`namma.navgrid`	5 cm occupancy grid, distance transform, walked-path repair, largest component	`navgrid.json`, `minimap.png`
7	`namma.export`	PLY to SPZ with splat-transform; scene manifest	`scene.spz`, `scene.json`
8	Web viewer	React, Three.js, Spark.js; Fuse.js search; A* in a Web Worker	Static site

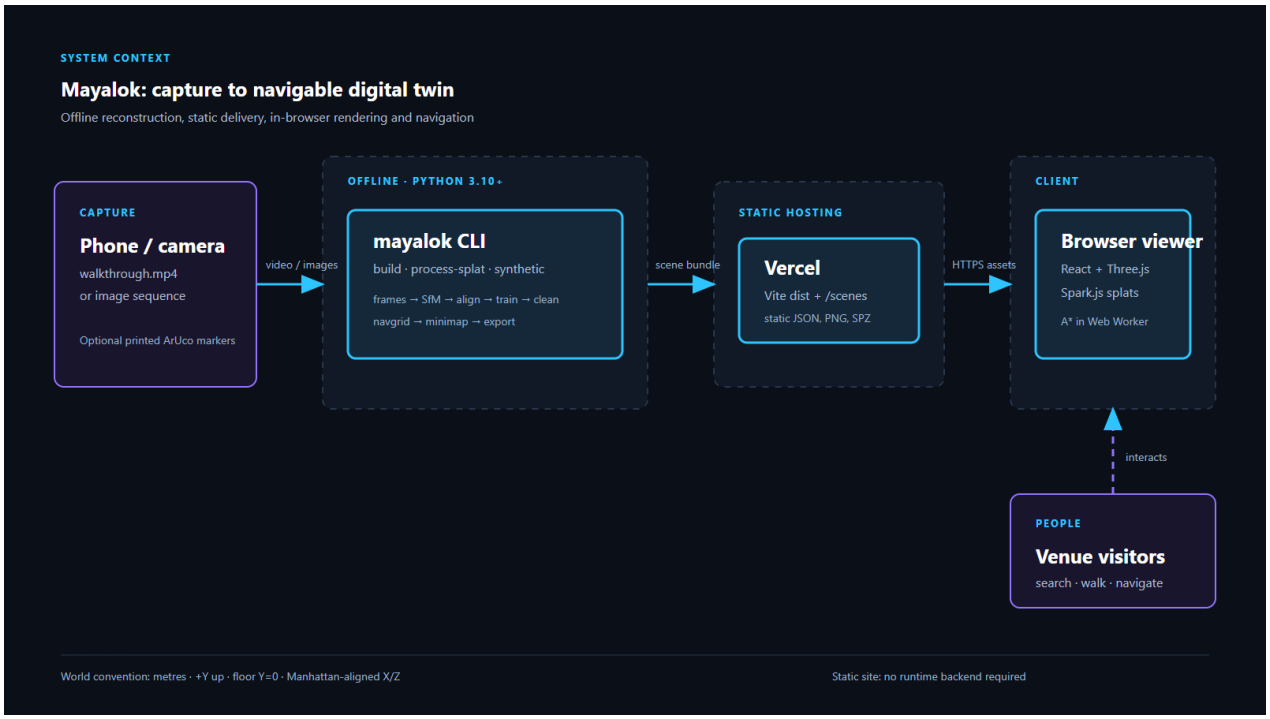


Fig. 2. System context from the repository diagrams: capture by phone, offline processing by the `mayalok` command-line tool, static hosting of the scene bundle, and the browser viewer used by venue visitors. The world convention is meters, $+Y$ up, floor at $Y = 0$, Manhattan-aligned X/Z.

optional vocabulary-tree loop detection. Poses come from COLMAP's global mapper (GLOMAP) [10]. If the registered fraction falls below 70%, the incremental mapper [9] is also run and the larger model is kept. We report registration ratio $\rho = N_{\text{reg}}/N_{\text{img}}$ and mean reprojection error

$$\bar{e} = \frac{1}{|\mathcal{O}|} \sum_{(i,j)\in\mathcal{O}} \left\| \pi(\mathbf{K}_i, \mathbf{T}_i, \mathbf{X}_j) - \mathbf{x}_{ij} \right\|_2 , \tag{4}$$

where $\mathcal{O}$ is the set of observations, π is the projection, and $\mathbf{x}_{ij}$ is the measured keypoint. Both quantities are standard but, as Section V shows, neither certifies global consistency.

E. Metric Scale, Gravity, Floor, and Yaw

SfM output has arbitrary scale and orientation. The "up" direction is the negated mean of the cameras' downward axes, since full walking loops cancel forward and backward tilt. The floor is the lowest significant dense layer of sparse points below the median camera height, refined by RANSAC [13] with tilt limited to 10° and a least-squares refit on the inliers. Scale uses the first available source in the order manual override, ArUco markers, phone height. For markers, each marker (15 cm, up to 400 frames inspected) is triangulated by the direct linear transform [11] and accepted only with at least three views and low reprojection error; the scale is

$$s = \frac{\ell_{\text{printed}}}{\text{median}_m \left(\hat{\ell}_m \right)}, \tag{5}$$

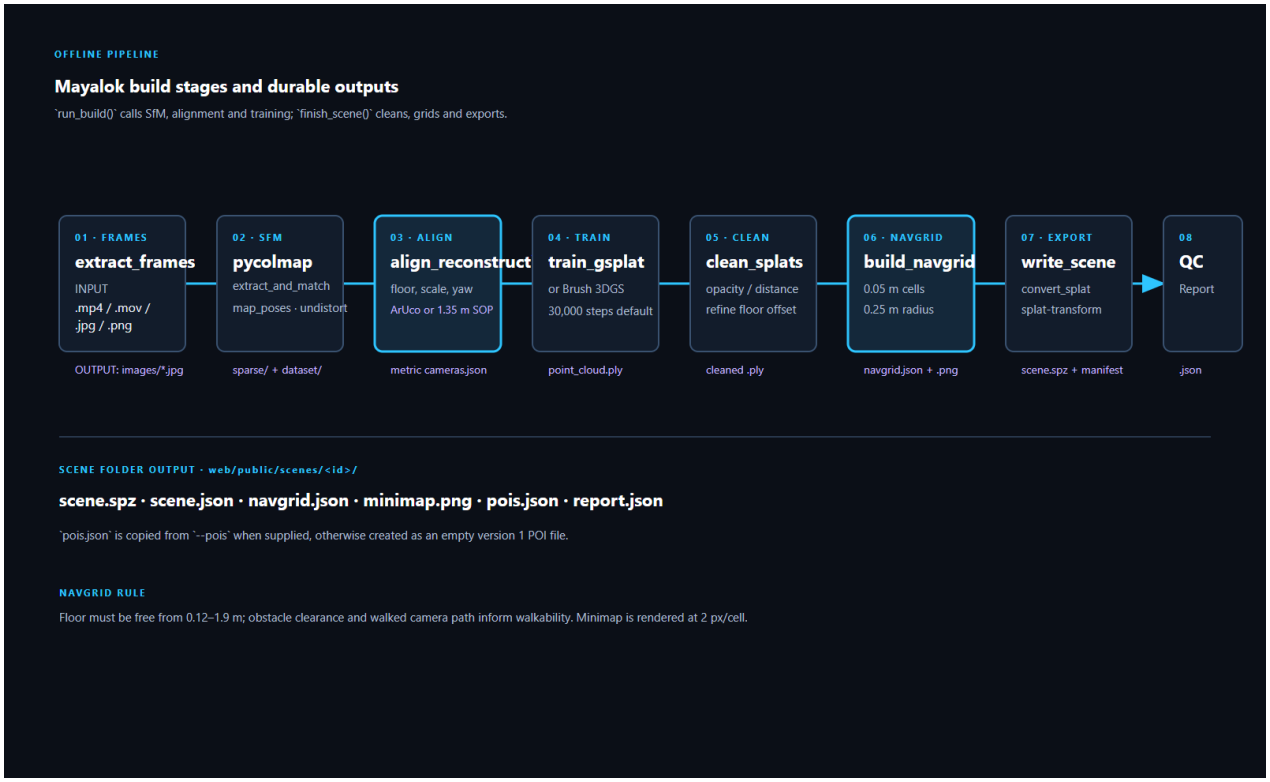


Fig. 3. Build stages and durable outputs. `run_build()` runs frames, SfM, alignment, and training; `finish_scene()` runs cleaning, the navigation grid, and export. The same tail also serves `process-splat`, which accepts an externally trained splat.

with $\ell_{\text{printed}} = 0.15$ m and $\hat{\ell}_m$ the reconstructed side length of marker m. Without markers, the phone-height rule gives

$$s = \frac{h_0}{\tilde{h}_{\text{cam}} - h_{\text{floor}}}, \tag{6}$$

where $h_0 = 1.35$ m, $\tilde{h}_{\text{cam}}$ is the median camera height along the up axis, and h_{floor} is the fitted floor height. Yaw is chosen to maximize the sum of squared histogram counts of wall points along the two horizontal axes over a 90° sweep, which aligns walls with the axes [14]. One similarity transform is applied to cameras and points, leaving the scene metric, $+Y$ up, floor at $Y = 0$, with the origin at the centre of the walked area. The incubation scene used the phone-height rule.

F. Gaussian Splat Training

The trainer is gsplat's reference script [6] with MCMC densification [5], 30,000 steps by default, and a hard cap of 1.5 million Gaussians. World-space normalization is disabled so the metric frame is preserved, and every 8th frame is held out so the trainer can report held-out image quality. On Windows machines where gsplat does not build, Brush [7] is used instead. Both Mayalok runs reported here used Brush: 20,000 steps for the single-video run and 25,000 steps for the all-captures run. Held-out image-quality metrics such as PSNR and SSIM [23] were not recorded for these runs, so we make no image-quality claim beyond visual inspection.

G. Cleaning

Four filters run in order, followed by a floor refit. First, Gaussians with opacity below 0.05 are removed. Second, Gaussians whose largest extent exceeds 1 m are removed. Third, Gaussians outside a bounding box that extends 3 m beyond the camera positions (and -0.5 to 4 m in height) are removed. Fourth, statistical outliers are removed: for each remaining Gaussian, let $\bar{d}_i$ be the mean distance to its $k = 8$ nearest neighbours; Gaussian i is dropped if

$$\bar{d}_i > \mu_{\bar{d}} + 3\,\sigma_{\bar{d}}. \tag{7}$$

The floor height is then refitted as the densest opaque layer within ± 0.3 m of $Y = 0$ and corrected by a vertical-only shift, so view-dependent color is unaffected.

H. Navigation Grid

The cleaned splat is sliced into a regular grid of 5 cm cells on the X–Z plane. Let α_i be the opacity and y_i the height of Gaussian i. A cell is an *obstacle* if the opacity summed over Gaussians with $0.12 < y_i < 1.9$ m (and $\alpha_i > 0.2$) is at least 1.5 and the cell belongs to a connected blob of at least four cells, which removes lone specks while keeping one-cell-thick walls. A cell has *floor* evidence if the summed opacity of Gaussians with $|y_i| < 0.08$ m is at least 1.0, followed by a small morphological closing to bridge texture gaps. Cells

traversed by the camera path are known to be free because the operator stood there. The free set is

$$\mathcal{F} = (\mathcal{C}_{\text{floor}} \setminus \mathcal{C}_{\text{obs}}) \cup \mathcal{C}_{\text{walked}}, \tag{8}$$

and only the part connected to the walked path is kept. With $d(c)$ the Euclidean distance from cell c to the nearest blocked cell, a cell is walkable if

$$c \in \mathcal{W} \iff (d(c) \geq r_{\text{agent}}) \vee (c \in \mathcal{C}_{\text{path}} \cap \mathcal{F}), \tag{9}$$

and the largest connected walkable component is retained. The default agent radius is $r_{\text{agent}} = 0.25$ m. A clearance map in centimeters (saturating at 255) is exported with the grid, serialized as base64 `uint8` row-major arrays in `navgrid.json`, and a top-down minimap is rendered from splat colors.

I. Export and Scene Bundle

The splat is converted to compressed SPZ with splat-transform [18], [21]. `scene.json` records the frame convention, bounds, spawn pose, camera trajectory, ArUco anchors (marker poses, kept for future AR use), and quality-control statistics. A scene folder is the complete deployable unit, and a venue is added by copying it under `web/public/scenes/<id>/` and registering it in `index.json`. An alternative route lets a user capture with a commercial app, export a PLY splat, and run `process-splat`, which skips stages 1–4.

J. Web Viewer

The viewer uses Vite, React 19, Three.js [20], and Spark.js [19]. First-person controls provide WASD movement, pointer-lock or drag look, and a noclip mode, with an eye height of 1.6 m and a walking speed of 1.6 m/s, both frame-rate independent. Collision uses the navigation grid. Figure 4 shows the runtime components. Places are stored in `pois.json`; Table IV gives the schema. Zone places (rooms) are drawn as tinted floor polygons, object places as labels with a stem to the floor, and `zoneAt(point)` returns the smallest zone containing a floor point so a counter inside a kitchen is found first. An admin mode (`?admin`) adds and edits places, with local storage and JSON import and export; there is no server-side save by design.

TABLE IV
POINT-OF-INTEREST RECORD (`POIS.JSON`, VERSION 1)

Field	Type	Meaning
`id`	string	Stable key; deep link `?to=<id>`
`name`	string	Label and search text
`category`	string	One of nine; unknown falls back to `other`
`tags`	string[]	Optional search keywords
`description`	string	Optional free text
`position`	[x,y,z]	Label anchor in metric frame
`zone`	[[x,z],...]	Optional floor polygon (3+ points)

K. Search

Search builds a Fuse.js index [22] over places with field weights for `name` (0.6), `tags` (0.25), `category` (0.1), and `description` (0.05), a match threshold of 0.4 for typo tolerance, and `ignoreLocation` so a match anywhere in a string counts. The interface returns at most eight results; an empty query returns the first places unfiltered. Pressing `/` focuses the box, Enter jumps to a place, and Shift+Enter starts navigation. Matching is lexical, so synonyms exist only if authors add them as tags.

L. Pathfinding and Instructions

Routing runs A* [15] over the walkable cells of the grid, 8-connected, with diagonal cost $\sqrt{2}$ and no corner cutting: a diagonal step requires both adjacent orthogonal cells to be walkable. The cost of stepping into cell v with base length $w_{uv} \in \{1, \sqrt{2}\}$ and clearance $\kappa(v)$ is

$$c(u \to v) = w_{uv} \left(1 + \lambda \max\left(0,\, 1 - \frac{\kappa(v)}{\kappa_0}\right)\right), \tag{10}$$

with preferred clearance $\kappa_0 = 0.6$ m and wall penalty $\lambda = 2$. A route that hugs a wall therefore pays up to $3\times$ the distance, so paths centre themselves in corridors and doorways. The heuristic is octile distance in cells,

$$h(r, c) = \Delta_r + \Delta_c + (\sqrt{2} - 2)\min(\Delta_r, \Delta_c), \tag{11}$$

which does not overestimate because the penalty only increases step costs. The open set is a binary min-heap with typed-array scores. Start and goal snap to the nearest walkable cell, and for a zone place the goal is the zone centroid. The cell path is smoothed by string pulling: points are dropped while the straight segment between the anchor and a candidate stays on walkable cells (sampled every half cell). The route begins exactly at the user's position when that spot is walkable.

Planning runs in a Web Worker. The router posts the grid once and then sends plan requests, so planning never blocks rendering; if a worker cannot be created, planning runs inline. While following a route, if the player drifts more than 1.6 m from the polyline (at least 1.5 s since the last plan and not during auto-walk), the route is re-planned from the current position.

At each interior vertex the signed turn angle

$$\theta = \operatorname{atan2}(\mathbf{a} \times \mathbf{b},\ \mathbf{a} \cdot \mathbf{b}) \tag{12}$$

between the incoming and outgoing headings $\mathbf{a}$ and $\mathbf{b}$ is classified: below 20° is straight (no instruction), 20°–50° slight left or right, 50°–135° left or right, and above 135° a U-turn. The instruction list is *start*, the turns with their distance along the route, and *arrive*. The HUD shows an arrow, text such as "Turn left in 6 m" ("now" within 1.5 m), the remaining distance, and an ETA at 1.2 m/s; arrival triggers within 0.9 m of the end. Auto-walk (key `F`) drives the camera along the route. Figure 5 shows the interaction from search to auto-walk.

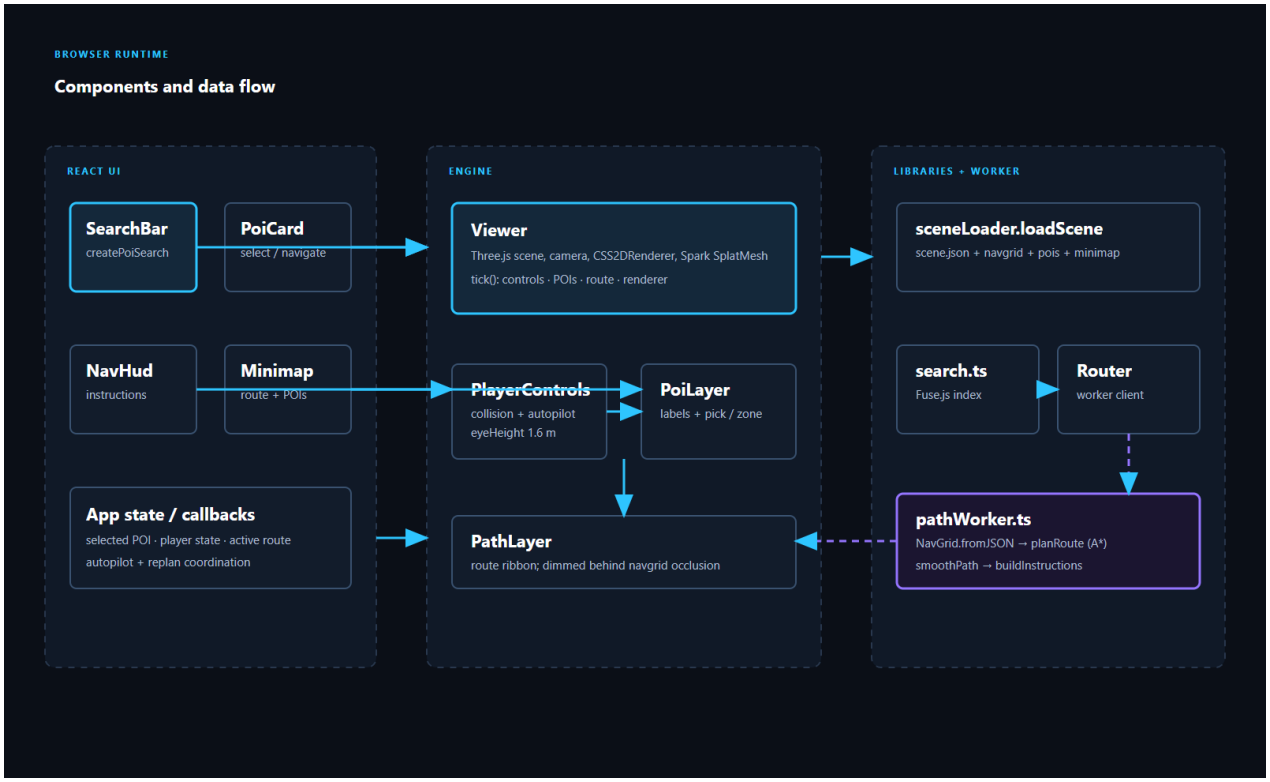


Fig. 4. Browser runtime components and data flow. The React user interface (search bar, place card, navigation HUD, minimap) talks to the engine (viewer, player controls, place and path layers), which uses the scene loader, the Fuse.js index, and a router that posts planning requests to the A* Web Worker.

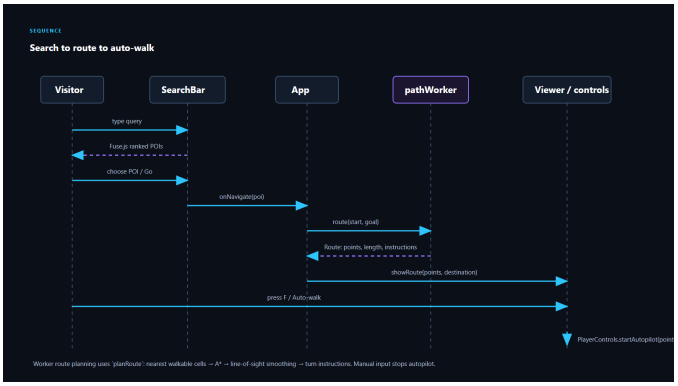


Fig. 5. Search-to-route-to-auto-walk sequence. A selected place triggers a plan request to the Web Worker, which returns points, length, and instructions for rendering; manual input stops auto-walk.

TABLE V
PARAMETER SUMMARY (DEFAULTS UNLESS MARKED)

Item	Value
Keyframe rate / long side	2 per s / 1600 px (run: 5.5 per s, 1024 px*)
Blur floor	35% of median sharpness
Camera model; SIFT features	`SIMPLE_RADIAL`; 8,192 max
Match window	15 neighbours
ArUco dictionary; side	`DICT_4X4_50`; 0.15 m
Phone height	1.35 m
Floor tilt limit	10°
Trainer; steps; cap	gsplat MCMC; 30k; 1.5M (run: Brush 20k, 25k*)
Cleaning	α<0.05; extent >1 m; k=8, 3σ
Grid; agent radius; band	5 cm; 0.25 m; 0.12–1.9 m
A* clearance, penalty	0.6 m, 2.0
Re-route; arrival	1.6 m drift; 0.9 m
Eye height; walk speed; ETA speed	1.6 m; 1.6 m/s; 1.2 m/s

M. Configuration Summary

Table V collects the main parameters. Values marked with an asterisk differ in the reported runs from the defaults, as noted in the text.

N. Evaluation Quantities

We report only quantities the pipeline records. The registration ratio ρ and mean reprojection error $\bar{e}$ measure SfM. The retention ratio after cleaning is

$$\eta = \frac{N_{\text{out}}}{N_{\text{in}}}, \tag{13}$$

the compressed-splat and grid sizes are in megabytes and kilobytes, and the walkable area is

$$A_{\text{walk}} = |\mathcal{W}| \cdot \delta^2, \tag{14}$$

with $\delta = 0.05$ m the cell size. We do not report novel-view PSNR, SSIM, absolute scale error, route-length accuracy, or user-task times, because those were not measured.

V. Experimental Results

A. Single-Video Run

Table VI reports the single-video reconstruction. Of 293 extracted keyframes, 65 registered ($\rho = 22\%$) in the largest of 7 sub-models, with 6,660 sparse points and a mean reprojection error of 0.52 px. SfM took about 155 s on CPU. The low registration ratio follows the capture conditions: a low-texture interior and fast pans meant most frames could not be linked to the main model. The consequence is a partial reconstruction of the room.

TABLE VI
Single-Video Run: SfM and Training

Quantity	Value
Input video (portrait)	56.8 s
Keyframes extracted	293
Registered images (largest of 7 sub-models)	65
Registration ratio ρ	22%
Sparse 3D points	6,660
Mean reprojection error $\bar{e}$	0.52 px
SfM wall time (CPU)	about 155 s
Trainer; steps	Brush; 20,000
Raw Gaussians	430,487

Brush trained 430,487 Gaussians for 20,000 steps on the laptop GPU. Table VII breaks down the cleaning stage from `report.json`. About 51.5% of the raw Gaussians were removed for low opacity, and a further 14,406 fell outside the bounds around the walked path; 191,378 remained, so $\eta = 44.5\%$. The floor refit applied a vertical shift of 0.285 m. The whole clean, grid, and export tail took 7.04 s.

TABLE VII
Single-Video Run: Cleaning, Grid, and Export (from `report.json`)

Quantity	Value
Input Gaussians	430,487
Removed, transparent ($\alpha < 0.05$)	221,679
Removed, oversized	230
Removed, outside bounds	14,406
Removed, isolated (statistical)	2,794
Output Gaussians	191,378
Retention ratio η	44.5%
Floor offset applied	0.285 m
Grid size (rows $\times$ cols, 5 cm)	140 $\times$ 95
Obstacle cells; floor cells	567; 138
Walkable area, shipped grid	2.66 m^2
Walkable area at 0.25 m radius	0.71 m^2
`scene.spz` size	4.7 MB
`navgrid.json` size	35.7 KB
Clean / grid / export time	0.86 s / 0.04 s / 6.14 s

B. Walkable Area and Agent Radius

The shipped grid was built with a reduced agent radius of 0.05 m (the `agentRadius` field of `navgrid.json`), which gives $A_{\text{walk}} = 2.66$ m^2. At the default 0.25 m the same scene yields only 0.71 m^2. The reason is visible in the construction. Only 138 floor cells (0.35 m^2) had reconstructed floor evidence, whereas the walkable set contains $2.66/0.05^2 = 1{,}064$ cells, so most walkable cells come from the walked-path repair rule, which frees a 0.3 m radius around the camera trajectory. That makes the free region a narrow strip, and inflating obstacles by 0.25 m leaves little of it. We therefore treat the 2.66 m^2 grid as a tuned setting that keeps routes possible in a partial scene, not as evidence of a fully mapped room, and the radius is a build-time parameter with no runtime override in the viewer. The scene is a corridor-like fragment, as the minimap in Fig. 6(a) shows.

C. All-Captures Run

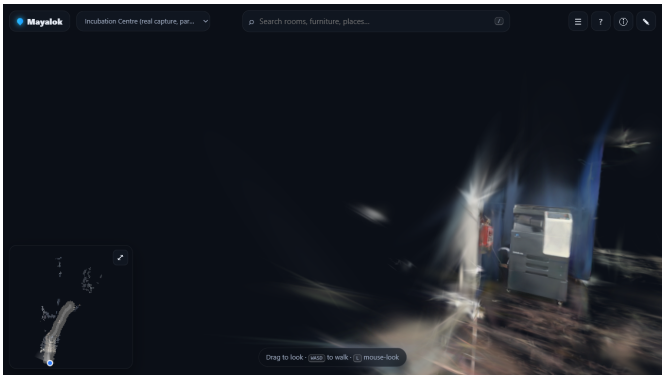
We also ran the pipeline on all 603 images (7 videos and 32 photos) of the same venue. Table VIII compares the two runs. GLOMAP registered 523 of 603 images (86.7%) in a single model, and 496 remained after outlier removal, with a mean reprojection error of 0.72 px. Brush trained 520,066 Gaussians for 25,000 steps. Judged on registration ratio and reprojection error, this run looks better than the single-video run. Visually it failed: the splat showed cross-clip misalignment, that is, content from different clips did not line up, and we excluded it from the shipped scene. The single-video scene is therefore the one packaged with the viewer.

TABLE VIII
Comparison of the Two Reconstruction Runs

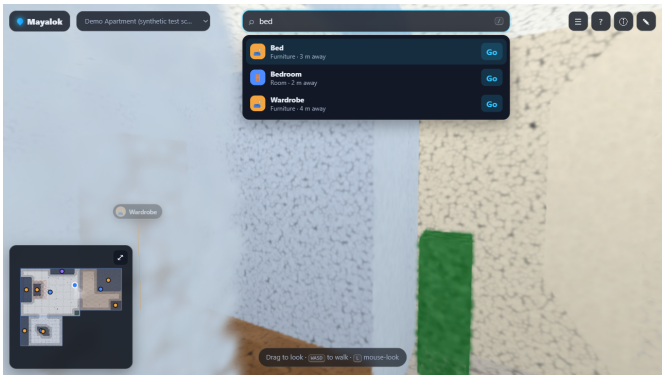
Item	Single video	All captures
Images	293 keyframes	603
SfM engine	pycolmap, 7 sub-models	GLOMAP, one model
Registered	65 (22%)	523 (86.7%); 496 after outlier removal
Mean reprojection error	0.52 px	0.72 px
Sparse points	6,660	not recorded here
Trainer; steps	Brush; 20k	Brush; 25k
Gaussians	430,487 raw; 191,378 clean	520,066
Outcome	Packaged as `incubation` scene	Visual cross-clip misalignment; excluded

D. Why Reprojection Error Did Not Predict Usability

The all-captures run is the most instructive result. Mean reprojection error is a local residual: it is small when each point projects close to its measured keypoint in the images that observe it. It does not constrain whether two clips that never share many observations are placed consistently. Registering 523 images into one model at 0.72 px can therefore coexist with a pair of clips offset from each other. We did not isolate the cause. Plausible, untested factors in this configuration are (i) separate intrinsics per clip, (ii) sequential matching, which links frames mainly within a clip and across clip boundaries only through neighbours in file order unless the optional vocabulary tree is enabled, and (iii) repetitive or low-texture surfaces that produce wrong cross-clip matches. These are hypotheses to test with a loop-closure matcher and ArUco markers visible in several clips, not findings. The practical lesson, which feeds the revised SOP, is to check a splat



(a) Real capture: the incubation-centre scene from one phone video, with its minimap (65 of 293 keyframes registered).



(b) Fuzzy search over tagged places on the synthetic `demo-apartment` scene, with category and live distance.



(c) Place card on the synthetic scene, offering Navigate and Go there.



(d) Expanded floor map of the synthetic scene with room zones and tagged places.

Fig. 6. Viewer screenshots. Panel (a) is the real, partial reconstruction; panels (b)–(d) use the procedurally generated `demo-apartment` test scene and demonstrate interface features, not reconstruction quality.

visually, and ideally against held-out views, before trusting registration statistics.

E. Viewer Output

Figure 6(a) shows the real incubation scene in the viewer, with the minimap in the lower left. The reconstruction is partial: a printer, a red object, and a blue partition are recognizable, while surrounding regions are dark or stretched into streaks where Gaussians are poorly constrained. Figures 6(b)–(d) show search, the place card, and the floor map on the `demo-apartment` scene, a procedurally generated three-room apartment used for development and tests. These three screenshots demonstrate the interface and routing features on synthetic data, not reconstruction quality; the `incubation` scene currently ships with an empty `pois.json`, so its places are still to be annotated.

F. Software Verification

The routing library has 11 passing unit tests (vitest, `astar.test.ts`) covering the A* search, turn instructions, and polygon geometry, and the pipeline has a pytest suite (alignment, navigation grid, SfM, splats and frames) that runs offline on a synthetic apartment generator. We do not quote coverage figures. Planning time in milliseconds is returned with each route and logged to the console, but we have no benchmark table for it, so we make no latency claim. The reconstruction pipeline is published as the `mayalok` package (`pip install mayalok`, PyPI 0.1.0).

VI. Discussion

The results support a focused claim. A single handheld smartphone video can produce, with about 155 s of CPU SfM and a Brush training run on a laptop GPU, a compact splat (4.7 MB) and a derived grid that the viewer can render and route over. The same evidence also shows the limits of that claim: only 22% of keyframes registered, the reconstruction covers part of one room, and metric scale rests on an assumed phone height rather than printed markers. The distances, route lengths, and ETAs the viewer reports are therefore only as accurate as that assumption.

The single-grid design has a practical advantage and a cost. Because collision, routing, line-of-sight dimming of labels and route ribbons, and the minimap all read the same grid, they are consistent with each other and cheap to compute in the browser. The cost is that obstacles come from splat density, so thin or transparent objects, glass, and mirrors can be missed or ghosted, and the grid is two-dimensional. The walked-path repair rule also trades safety for completeness: it keeps routes

available in a texture-poor floor, but it can mark as free a cell the camera passed over without ever observing the floor there.

The failed all-captures run is a more useful outcome than a clean success would have been. It shows that a pipeline whose summary statistics look good can still emit an unusable splat, which is why each scene ships its build statistics in `report.json` and why the revised SOP asks for slower pans, more overlap, and printed markers. We do not claim the 523-image result is wrong, only that it was visually unusable for our purpose.

VII. Threats to Validity

Single venue and single run per configuration. All numbers come from one capture session. We cannot say how registration, cleaning, or grid statistics vary across venues, lighting, or phones, and we report no confidence intervals.

Metric scale. Scale for the shipped scene comes from the 1.35 m phone-height rule, not ArUco markers, and we did not measure a ground-truth distance in the venue. The scale error is unknown.

No image-quality metrics. Held-out PSNR and SSIM were not recorded for either run, and visual assessment of the failed run is qualitative.

Tuned grid. The shipped walkable area uses a 0.05 m agent radius rather than the 0.25 m default, and most walkable cells come from the walked-path repair rule. The grid should not be read as an independent measurement of free floor.

No user study or latency benchmark. We did not test the viewer with users, and we did not tabulate planning time or frame rate.

Synthetic demonstrations. The search, place-card, and map screenshots use a procedurally generated scene. They show interface behavior, not reconstruction fidelity.

Documentation drift. Some project documents describe defaults (30,000 steps, gsplat) that differ from the settings used in the reported runs (Brush, 20,000 and 25,000 steps). We report the run settings.

VIII. Limitations and Future Work

Known limitations. The grid is single-floor, so stairs, ramps, and multi-level venues are not modelled. Dynamic obstacles (people, moved furniture) are not detected. Splats carry no explicit surface mesh. Mirrors, glass, and blank walls degrade SfM and training, and training needs a CUDA GPU (a cloud notebook is the fallback). Search is lexical. Admin edits are browser-local until exported. There is no multi-user editing.

Near-term work. The immediate extension is a re-capture of the incubation centre that follows the revised SOP, with printed ArUco markers visible in several clips and vocabulary-tree loop detection enabled, to test the hypotheses in Section V. We would then record held-out PSNR and SSIM, measure a known distance to quantify scale error, and tabulate planning latency.

Longer-term work. The planned augmented-reality layer would localize a phone camera against the splat and overlay the route on the live camera feed, using the floor markers already exported as anchors in `scene.json`. Further directions are semi-automatic place suggestion from the splat, incremental re-capture of changed zones, and multi-floor stitching.

IX. Conclusion

This report presents Mayalok, a pipeline and static web viewer that turns smartphone video into a metric Gaussian-splat digital twin with a 5 cm walkable grid, fuzzy place search, and wall-aware A* routing in the browser. On one real incubation-centre capture, a single 56.8 s video produced 65 registered keyframes out of 293, a 191,378-Gaussian cleaned splat, a 4.7 MB `scene.spz`, and a 2.66 m^2 walkable area after agent-radius tuning (0.71 m^2 at 0.25 m). A 603-image run registered 523 images at 0.72 px yet failed visually and was excluded, showing that registration statistics alone do not certify a usable splat. The main value of this version is clarity about what is demonstrated, a working end-to-end system with a partial real scene, and what remains to be validated, namely scale accuracy, image quality, multi-clip consistency, and user-facing performance.

Acknowledgment

The authors thank the open-source communities behind COLMAP and GLOMAP, gsplat, Brush, Three.js, Spark.js, Fuse.js, and splat-transform, whose tools made this system possible, and the Techfest 2026–27 Namma Space challenge organizers at IIT Bombay.

References

[1] B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng, "NeRF: Representing scenes as neural radiance fields for view synthesis," in *Proc. European Conf. Computer Vision (ECCV)*, 2020, pp. 405–421.

[2] J. T. Barron, B. Mildenhall, D. Verbin, P. P. Srinivasan, and P. Hedman, "Mip-NeRF 360: Unbounded anti-aliased neural radiance fields," in *Proc. IEEE/CVF Conf. Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 5470–5479.

[3] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3D Gaussian splatting for real-time radiance field rendering," *ACM Transactions on Graphics (Proc. SIGGRAPH)*, vol. 42, no. 4, pp. 139:1–139:14, 2023.

[4] Team QWERTY, "Mayalok: Smartphone walkthrough to a searchable 3D digital twin with in-browser navigation," https://github.com/The-Harsh-Vardhan/mayalok, live demo: https://mayalok.vercel.app. Accessed: Nov. 2026.

[5] S. Kheradmand, D. Rebain, G. Sharma, W. Sun, Y.-C. Tseng, H. Isack, A. Kar, A. Tagliasacchi, and K. M. Yi, "3D Gaussian splatting as Markov chain Monte Carlo," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 37, 2024.

[6] V. Ye, R. Li, J. Kerr, M. Turkulainen, B. Yi, Z. Pan, O. Seiskari, J. Ye, J. Hu, M. Tancik, and A. Kanazawa, "gsplat: An open-source library for Gaussian splatting," arXiv preprint arXiv:2409.06765, 2024.

[7] A. Brussee, "Brush: A 3D Gaussian splatting engine," https://github.com/ArthurBrussee/brush, accessed: Nov. 2026.

[8] D. G. Lowe, "Distinctive image features from scale-invariant keypoints," *International Journal of Computer Vision*, vol. 60, no. 2, pp. 91–110, 2004.

[9] J. L. Schönberger and J.-M. Frahm, "Structure-from-motion revisited," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 4104–4113.

[10] L. Pan, D. Baráth, M. Pollefeys, and J. L. Schönberger, "Global structure-from-motion revisited," in *Proc. European Conf. Computer Vision (ECCV)*, 2024.

[11] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge, U.K.: Cambridge University Press, 2004.

[12] S. Garrido-Jurado, R. Muñoz-Salinas, F. J. Madrid-Cuevas, and M. J. Marín-Jiménez, "Automatic generation and detection of highly reliable fiducial markers under occlusion," *Pattern Recognition*, vol. 47, no. 6, pp. 2280–2292, 2014.

[13] M. A. Fischler and R. C. Bolles, "Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography," *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981.

[14] J. M. Coughlan and A. L. Yuille, "Manhattan world: Compass direction from a single image by Bayesian inference," in *Proc. IEEE Int. Conf. Computer Vision (ICCV)*, vol. 2, 1999, pp. 941–947.

[15] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

[16] P. Virtanen *et al.*, "SciPy 1.0: Fundamental algorithms for scientific computing in Python," *Nature Methods*, vol. 17, pp. 261–272, 2020.

[17] G. Bradski, "The OpenCV library," *Dr. Dobb's Journal of Software Tools*, 2000.

[18] Niantic, Inc., "SPZ: A compressed file format for 3D Gaussian splats," https://github.com/nianticlabs/spz, accessed: Nov. 2026.

[19] Spark.js contributors, "Spark: A Gaussian splat renderer for Three.js," https://github.com/sparkjsdev/spark, accessed: Nov. 2026.

[20] three.js authors, "three.js: JavaScript 3D library," https://threejs.org, accessed: Nov. 2026.

[21] PlayCanvas, "splat-transform: Command-line tool for converting Gaussian splat formats," https://github.com/playcanvas/splat-transform, accessed: Nov. 2026.

[22] K. Risk, "Fuse.js: Lightweight fuzzy-search library," https://www.fusejs.io, accessed: Nov. 2026.

[23] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004.